

CHANNEL Z0 – NODES & FAILOVER (STATION BULLETIN Nº 5)

CONTENTS

▶ Read this first: a linear channel is a singleton	2
▶ The topology	3
▶ Standing up a node	4
The two flags that decide whether the node is a station or an empty box	5
Prove the tools before you trust them with a machine	6
The channel number is not 1	7
▶ How exactly-one-on-air works	7
▶ Prove it, don't assume it	8
▶ The one way to get this wrong	9
▶ Running a second channel	10
▶ Known limits, plainly	11

CHANNEL Z0 · CH 0 · DESIG RL-Z0
MASTER CONTROL – REDUNDANT
one lease, one transmitter, several machines

How to run the station on more than one machine: what that actually buys you, what it can't, and how to stand a node up in one command.

SEC.01 READ THIS FIRST: A LINEAR CHANNEL IS A SINGLETON

The instinct with "add more machines" is load balancing, and for a television channel that instinct is mostly wrong. It's worth being precise about why, because the design follows from it.

A channel is one timeline. At 19:04 the station is four minutes into Ground Zero, everywhere, for everyone. Two playout machines encoding the same schedule do not produce one channel twice – they produce two **different** channels that drift apart within seconds, because ffmpeg timing, filler selection, and encoder state are not deterministic across machines.

Two publishers corrupt the stream. The tower accepts one RTMP publisher per stream key. Point two at it and Owncast gets interleaved keyframes from two unrelated encodes. The failure is not "slightly degraded" – it's a channel that won't play.

The viewer-scaling problem is already solved, elsewhere. The station's whole architecture is **one stream leaves the house, the VPS does the crowd work**. Ten viewers or ten thousand, the playout PC uploads exactly one ~5 Mbps stream. Adding playout nodes does nothing for audience size, because audience was never the bottleneck.

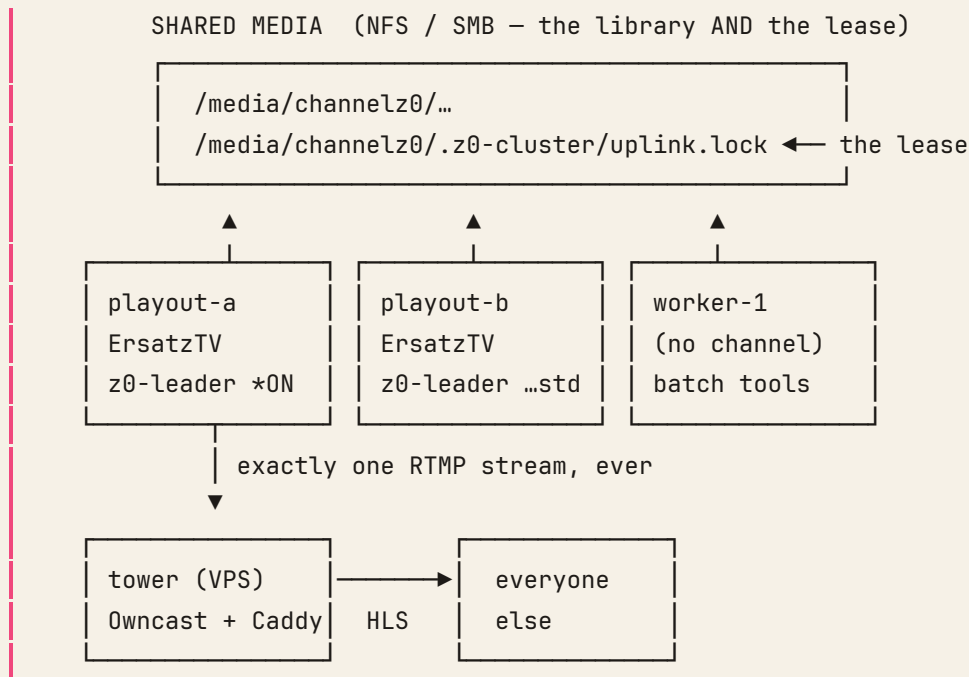
So: **you cannot load-balance playout**. What you **can** do:

GOAL	DOES ADDING NODES HELP?	HOW
Survive a dead payout box	Yes – the main reason to do this	Active/standby with a lease. Below.
Serve more viewers	No – already handled	The tower. See build-guide ^{RL-Z0-BUILD-GUIDE} Phase 1.
Run a second channel (Z0-2)	Yes – real horizontal scale	One node per channel, separate stream keys.
Faster library work	Yes	worker nodes running the batch tools.
Zero-downtime failover	Not really	Expect a few seconds of dead air. See the timing below.

Everything below is about the first, third and fourth rows.

SEC.02

THE TOPOLOGY



Both playout nodes run ErsatzTV all the time – idle ErsatzTV costs almost nothing, and it means the standby is warm. Only the lease holder runs the uplink.

SEC.03

STANDING UP A NODE

One script, three roles. It inspects the machine, tells you what it found, prints a plan, and only then changes anything.

BASH

```
# Look, change nothing. Run this first on any new box.
tools/bootstrap-node.sh --check
```

```
# The tower (do this one first – the playout nodes need its domain)
tools/bootstrap-node.sh --role tower \
  --watch-domain watch.ch0.example --site-domain ch0.example --yes

# First playout node – with the station and the library it needs to air anything
tools/bootstrap-node.sh --role playout --node-id playout-a \
  --watch-domain watch.ch0.example --stream-key "$KEY" \
  --station-image /tmp/z0-station.tgz \
  --media-peer root@nas:/mnt/main-data/channelz0 \
  --media /mnt/z0 --yes

# Second playout node – same command, different id, SAME shared media
tools/bootstrap-node.sh --role playout --node-id playout-b \
  --watch-domain watch.ch0.example --stream-key "$KEY" \
  --station-image /tmp/z0-station.tgz \
  --media /mnt/z0 --nfs nas:/export/z0 --yes
```

`--dry-run` prints every action without taking it. It's idempotent – re-run it after changing a flag and it reconciles, backing up any existing `.env` first, since that file holds the one secret.

The only thing it won't do quietly is install Docker: that means piping a remote script into a shell, so it tells you the exact command and waits for `--install-docker` (or `-yes`).

THE TWO FLAGS THAT DECIDE WHETHER THE NODE IS A STATION OR AN EMPTY BOX

`--station-image` is the difference between "ErsatzTV is running" and "the channel is on air". Everything that makes this station **this station** – the FFmpeg profile, the libraries, the smart collections, the filler presets, the channel and its watermark, the graphics elements, the playout – is configured through the web UI and stored in one SQLite file. There is no config API: `/api/...` serves the SPA, not JSON. So the only faithful way to reproduce a station is to carry its database across.

BASH

```
# On the machine that is on air (read-only; safe against a live station):
tools/z0-station-image.sh --capture \
```

```
--from root@vile:/mnt/solid-state/ersatztv --out /tmp/z0-station.tgz
tools/z0-station-image.sh --inspect /tmp/z0-station.tgz
```

The image is a few megabytes. Restoring it re-points three things at the new machine, and each one is a trap if you skip it:

IT REWRITES	BECAUSE OTHERWISE
the FFmpeg profile's hardware accel and its name	the profile says h264_nvenc on a VAAPI box, silently falls back to software, and eats a CPU
Plex / Jellyfin / Emby sources – deleted	the new node inherits dead libraries pointing at a host it cannot reach. None of Z0's smart collections use them; every one is type:other_video against the local library
the playout anchor – cleared	an anchor stores an instruction index into the flattened schedule. Restored elsewhere it still looks valid and points at a different instruction. The week doesn't fail, it resumes mid-block and the only symptom is a day that looks subtly wrong

It also drops the search index, because a stale index makes every tag query resolve to nothing while the folders all look correctly full.

-media-peer mirrors the library. Reach for `fetch-archive.sh` here and you will get **a** library, not **this** one: it runs search queries, so a rebuild returns different titles with different tags, and its manifest only ever skips what is already down – it cannot replay a library. About a fifth of the station has no recipe at all (schoolroom, the Canadian and BC sets, the music cards, everything generated). Copy it:

BASH

```
tools/z0-media-sync.sh --from-peer root@vile:/mnt/main-data/channelz0 \
  --bwlimit 20000          # the station is broadcasting over this same line
tools/z0-media-sync.sh --from-peer root@vile:/mnt/main-data/channelz0 --verify
```

It never copies `.z0-cluster`. That directory **is** the lease – copy it and the new

box starts up believing it is already on air, which is the exact double-publish everything below exists to prevent.

PROVE THE TOOLS BEFORE YOU TRUST THEM WITH A MACHINE

BASH

```
| tools/test-spawn.sh      # ~5 seconds, no station, no media, no GPU
```

It builds a synthetic station in a scratch database and asserts the properties that decide whether a spawned node airs the right thing: the encoder is re-pointed, the remote libraries don't travel, the playout anchor is cleared, the index is dropped, the channel number is read rather than assumed, and the lease is never copied.

THE CHANNEL NUMBER IS NOT 1

ErsatzTV publishes at `/iptv/channel/<number>.ts`, and the number is whatever the station was built with. **Channel Z0 is number 0** – `/iptv/channel/1.ts` returns 404 on it. Bootstrap reads the real number out of the station image and writes it into `.env`; it used to hardcode 1, which produced a node that passed every health check and published nothing.

That is also why bootstrap's verification `ffprobes` the channel for decodable video instead of accepting an HTTP 200 from ErsatzTV. ErsatzTV answers on `:8409` with no libraries, no channel, and nothing to air – and it answers while wedged. The failover logic already knew this; a node `ffprobes` itself before standing for election precisely so a broken node doesn't win and broadcast silence.

SEC.04

HOW EXACTLY-ONE-ON-AIR WORKS

`playout/z0-leader.sh` wraps the uplink. Every playout node runs it; at most one publishes.

The lease. A directory on shared storage, `.z0-cluster/uplink.lock`. `mkdir` is atomic and fails if it exists – that's acquisition, with no window where two nodes both

believe they won. The holder writes a heartbeat into it every few seconds.

Taking over. A standby watches the heartbeat. If it stops changing for `TTL`, the standby steals the lease by `mv`-ing the lock directory aside – `rename` is atomic, so if two standbys pounce at once exactly one wins and the loser retries next tick.

Fencing, which is the part that matters. The old leader must be **off air** before the new one starts:

```
t=0      leader loses shared storage / dies / is partitioned
t=12s    FENCE – leader can't renew, so it kills its own uplink
t=20s    TTL – a standby steals the lease and goes on air
-----
          8s    dead air, and 8s of margin against double-publishing
```

`TTL > FENCE` is the whole safety argument. Eight seconds of dead air beats two stations at once. If you shorten these (`Z0_LEASE_*` in `.env`), keep the gap.

Clocks are not trusted. Comparing a heartbeat's `mtime` against the local clock breaks the moment two machines disagree about the time, which on a homelab is whenever NTP hiccups. Instead each node watches for the heartbeat **value to change** and times that with its own elapsed seconds. Skew between nodes is irrelevant. (Run NTP anyway, for legible logs.)

Sick nodes don't stand for election. Before taking the lease a node `ffprobes` its own channel URL – proving ErsatzTV is emitting decodable video, not merely answering HTTP. A node whose ErsatzTV is wedged stands down and lets a standby take it, rather than winning the election and airing silence. Standbys probe every tick; the leader probes every `Z0_HEALTH_EVERY` (30 s) while healthy and every tick once a probe fails, because each probe is a channel session ErsatzTV logs, and probing every 5 s buried the real log lines.

Planned reboots are fast. On `SIGTERM` the leader releases the lease deliberately instead of letting it time out, so `docker compose down` hands over in a second or two rather than the full TTL.

SEC.05

PROVE IT, DON'T ASSUME IT

Redundancy you haven't tested isn't redundancy.

BASH

```
tools/test-failover.sh          # 3 simulated nodes; ~30 seconds
Z0_TEST_NODES=5 tools/test-failover.sh
```

It runs real supervisor processes against a scratch cluster directory with a fake publisher, hard-kills the node that's on air (SIGKILL to its whole process group – a simulated power cut), and asserts both properties:

- ▶ **SAFETY** – never more than one publisher, sampled continuously throughout
- ▶ **LIVENESS** – a standby takes over within the TTL

Measured on the compressed test timings (renew 1s / fence 3s / ttl 5s): hard failure recovered in ~4.7 s, graceful handover in ~2.0 s, and the safety invariant held. No tower, no ErsatzTV, no media required – safe on a laptop.

Check the real cluster any time, from any node:

BASH

```
playout/z0-leader.sh --status
```

```
lease holder: playout-a
publishing:
  playout-a  since 2026-08-06T21:53:53Z
```

NODE	ROLE	STATE	LAST-SEEN	HOST
playout-a	playout	leader	2026-08-06T21:53:57Z	nuc-1
playout-b	playout	standby	2026-08-06T21:53:57Z	nuc-2

Two entries under `publishing:` means the thing this whole design exists to prevent is happening. `-status` shouts about it.

SEC.06

THE ONE WAY TO GET THIS WRONG

Every `playout` node must see the same lease. If `Z0_CLUSTER_DIR_HOST` points at local disk on each machine, both nodes will find an empty cluster, both will conclude they're alone, and both will publish. You will have built the exact failure the lease exists to prevent, and the symptom – a channel that won't play – looks nothing like the cause.

Put the cluster directory on the shared mount. `bootstrap-node.sh` warns when the media root looks local; believe it. `-status` from a second node is the quick confirmation: if it can't see the first node, they aren't clustered.

SEC.07

RUNNING A SECOND CHANNEL

This is the real horizontal scale, and it needs no coordination at all – a second channel is a second **station**, sharing only the media library:

1. A second stream key on the tower (Owncast: a second instance, or PeerTube – see `vps/peertube/`).
2. A node with its own `.env`: different `Z0_NODE_ID`, different `Z0_STREAM_KEY`, different `Z0_CHANNEL_URL` (ErsatzTV channel 2), and a **different** `Z0_CLUSTER_DIR_HOST` – channel 2's lease must not collide with channel 1's.
3. Give each channel its own standby if you want the same redundancy.

Media is read-only to playout, so any number of nodes can share one library.

SEC.08

KNOWN LIMITS, PLAINLY

- ▶ **Failover is not seamless.** Viewers see a few seconds of dead air and their player reconnects. Making that invisible needs a hot standby encoder and a switcher at the tower – a different, much larger project.
- ▶ **The standby resumes the schedule, not the frame.** ErsatzTV starts the new node at the current wall-clock position in the schedule, so you rejoin the programme roughly where you left it, not exactly.
- ▶ **Shared storage is a single point of failure.** The lease lives on the NAS; if the NAS dies, every node fences itself and the channel goes dark. That is the **correct** behaviour (better dark than doubled), but it means the NAS is now the thing to make redundant.
- ▶ **Split-brain across a network partition is bounded, not impossible.** Two nodes that can't see each other but **can** both reach the tower will, in the worst case, briefly overlap around the fence/TTL boundary. The margin exists to make that window small, not to make it zero.
- ▶ **No automatic fallback to a standby's media.** Nodes share one library. A node with a stale local copy will air the wrong thing without complaining.

If the transmitter falls over, another one picks it up. That's the whole idea. The bars are still there if both of them go.

SOURCE docs/clustering.tex SHA cc5ab7731230 BUILT 2026-09-15